Week 19 | Tokenization and noise-aware sensor design

Decide what information a token preserves

Week 7.3 used patches for masked reconstruction; Week 5 used a low-rank basis and selected sensors. Here the student compares those representations before training another network. The central question is whether errors arise from discarded field information, poorly conditioned observations or a learned map. These causes require different remedies.

The field is a simulated transverse-velocity array with shape 32 by 78. Point tokens, patches and a POD state vector are all legal constructions, but they present different sequence lengths and different meanings to attention. An architecture comparison that changes tokenization, rank, sensor noise and training budget simultaneously cannot attribute its result to attention alone.

Three representations of the same snapshot

Point tokenization assigns one token to each grid location. It yields 2496 tokens; coordinates identify physical location. A 4 by 6 patch partition exactly tiles the domain into 8 by 13 patches, hence 104 tokens with 24 scalar entries each. Patching and its inverse are exact rearrangements until an embedding compresses those entries.

A rank-eight POD representation yields eight coefficients for the entire snapshot. In the temporal decoder, those eight numbers form one token at a particular time, not eight unrelated time steps. Over a four-frame history, the attention sequence therefore has four tokens. Writing down both the token count and the feature dimension prevents a common factor-of-eight interpretation error.

Representation	Tokens per snapshot	Raw features per token	Invertibility
Grid points	2496	value plus coordinates	Exact before projection
4 by 6 patches	104	24 values plus position	Exact rearrangement
POD state vector	1	8 coefficients	Approximate after truncation

Computational cost is part of the contract

For B batches, H heads and N tokens, a stored float32 attention-score array uses 4 B H N squared bytes. One batch and four heads on point tokens requires about 99.7 million bytes just for scores, before gradients and other activations. Patch scores require about 173 thousand bytes. Their ratio is 576 because the token-count ratio is 24 and the score cost is quadratic.

$$\text{bytes}_{scores} = 4BHN^2$$

This is a storage accounting exercise, not a benchmark of a fused attention kernel. Optimized implementations can avoid materializing all scores. Actual wall time also depends on feature width, batch size, hardware and kernel choice. Students should report the analytical count separately from a measured run.

POD, projection and the unavoidable floor

Form a training snapshot matrix, subtract its training mean, and compute an orthonormal rank-r spatial basis. Project a new snapshot onto that frozen basis. The difference between the snapshot and its projection is representation error; no model whose decoder stays inside this affine subspace can remove it.

$$a = \Phi^T (u - \bar{u}), \qquad u_r = \bar{u} + \Phi a$$

Retained energy is the sum of the first r squared singular values divided by the sum over all available singular values. Reading the final entry of a cumulative-energy array reports the full basis energy, usually one, and says nothing about the rank actually used. The revised representation stores the rank-specific quantity directly.

The basis sign convention makes the largest-magnitude component of each mode positive. That removes a simple isolated-mode sign ambiguity but cannot eliminate arbitrary rotations in a nearly degenerate eigenspace. Saved checkpoints therefore include the exact mean, basis and coefficient scale. They must not rebuild a new basis when loaded on another platform.

Reconstruct from sparse observations

Let C select observed grid entries. After subtracting the same training mean, observations satisfy y approximately equal to C Phi a. Gappy POD solves a least-squares problem for a. It should use a numerical least-squares solver, not an explicit inverse, because conditioning can amplify observation noise.

$$\hat{a} = \arg\min_a \| C\Phi a - (y - C\bar{u}) \|_2^2$$

In a two-mode example, sensors giving rows (1,0) and (0,1) identify both coefficients directly. Rows (1,0) and (1,0.001) nearly duplicate the first measurement, making the second coefficient highly sensitive to noise. Two sensors are enough by counting unknowns, yet the geometry is still poor. Rank and conditioning answer different questions.

The perturbation equation is delta a approximately equal to the pseudoinverse of C Phi applied to observation noise. A small singular value in the observed basis therefore creates a large sensitivity direction. Plotting condition number alongside reconstruction error helps students see why a learned nonlinear branch cannot recover information that the observations never supplied reliably.

QR selection and noisy comparison

Pivoted QR selects an initial well-conditioned set from training modes. When the requested sensor count exceeds basis rank, the implementation uses a greedy D-optimal oversampling criterion rather than treating arbitrary residual QR pivots as optimized extra sensors. Sensor choice never uses evaluation field values.

The worked exercise compares sixteen selected sensors with sixteen geometry-lattice sensors, using the same noise vector and a noise standard deviation defined as one percent of the global training-field standard

deviation. This definition differs from one percent of each sensor's standard deviation. The chosen convention is recorded because otherwise two nominally one-percent experiments can supply different information.

The comparison uses a fixed field for illustration. A scientific estimate requires multiple fields and noise realizations, and a result from the old Week 5 protocol is contextual rather than directly interchangeable when its noise convention or basis differs. The full Week 20 release uses all retained evaluation frames and all declared model seeds.

Laboratory deliverables

The four tasks invert patching, calculate a frozen-basis representation floor, implement the gappy solve and audit noisy sensor designs with reconstruction, representation floor, rank and condition diagnostics. The noiseless gappy test reconstructs a synthetic vector lying exactly in the training subspace; that controlled case separates an implementation bug from unavoidable approximation error on CFD.

Submit an exact patch round trip, a three-panel reference/reconstruction/error figure with a separate error color scale, a sensor-condition table and a memory calculation. Explain why improving retained energy need not improve conditioning, and why the evaluation field cannot be used to choose the rank or sensors after its score is seen.

Reading and connection

Use the existing Week 5 modal laboratory and data contract for the course's established sensing conventions. Manohar et al., Data-Driven Sparse Sensor Placement for Reconstruction (2018), https://doi.org/10.1109/MCS.2018.2810460, supplies the primary sparse-sensing context. The present QR/D-optimal procedure is an educational implementation with its own declared observation model.

Week 20 keeps this information contract visible while adding learned models. A Transformer is tested where a set representation has an identifiable structural use: observations can be missing, and coordinates must remain attached to their values.

Printable laboratory worksheet

Task 1: Invert patching

Implement the inverse of the demonstrated 4x6 patch layout.

Interface contract: Input tokens:[patches,24] uses the demonstrated row-major 4-by-6 layout; shape is (height,width), both divisible by those patch sizes. Return the NumPy field in its original spatial order.

Task 2: Representation floor

Compute the relative error of projecting a field into the frozen basis. Do not fit a new basis on the evaluation field.

Interface contract: rep is a fitted lab.Representation; field is one spatial NumPy vector or a two-dimensional field of the fitted spatial size. Return the scalar global Frobenius relative L2 error of its orthogonal reconstruction, using the same truth norm.

Task 3: Gappy solve

Recover modal coefficients from a supplied subset of observations using least squares, not matrix inversion.

Interface contract: y is a vector of sampled values at ids. Return a full spatial vector from a least-squares solve of representation.modes[ids] against y-representation.mean[ids], then decode in the original affine basis.

Task 4: Audit sensor reconstruction against its floor

Given a representation, flattened truth field, sensor-index designs and optional observed sensor values, return reconstruction, relative field error, representation floor, sensor matrix rank and condition number for each design. Compare full-rank and rank-deficient designs and repeat with noisy observations; report observation error separately from representation error.

Interface contract: representation is fitted; truth is a nonzero spatial vector. designs is a list of nonempty unique integer sensor-index arrays. observations is None (use truth at sensors) or a parallel list of sampled vectors including any noise. Return a list in design order of dictionaries: prediction (full spatial NumPy vector), field_error (relative L2 to truth), floor (relative L2 of truth projection), rank (rank of sensed mode matrix), condition (2-norm condition number of that same sensed matrix). Duplicate/empty sensor designs and zero truth norm must raise ValueError.

Before submitting

Restart the kernel and run the worked examples and completed tasks. Confirm that all four coding checks pass and that no retained data or results file changed. Include the requested interpretation, identify every quantity fitted from training data, and separate test observations from decisions made using validation. Record unresolved budget or representation limits rather than removing inconvenient runs.

Assessment and worked interpretation

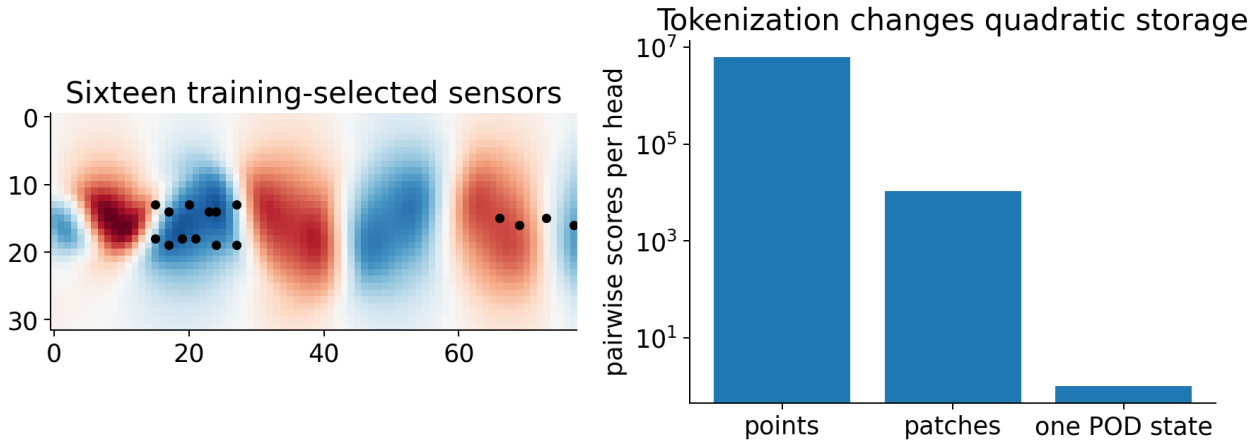
Coding: Tasks 1–3 carry 5 points each; the integrated Task 4 carries 25. Derivation (25): derive the orthogonal representation floor and the gappy least-squares solve. Interpretation (25): give evidence and limitations. Reproducibility (10): identify inputs, split, checkpoint, environment and every failed seed. Total 100. Passing function checks earns only the coding allocation.

Worked full-credit reasoning: A lower POD floor does not guarantee better reconstruction from noisy sparse sensors. More retained modes can make the sampled basis ill conditioned; compare the floor, noise amplification and field error with frozen sensor coordinates.

Misconception to reject: “Adding POD modes always improves a noisy sparse-sensor reconstruction.” Explain the failure in one equation or counterexample. An observation without protocol and limitation earns at most half of interpretation credit. Instructor judgment and a timed student pilot remain necessary.

Timing estimate: a 120-minute guided lab and roughly 3–5 hours including preparation and written analysis. This is an editorial estimate; a timed student pilot remains open.

Executed evidence and interpretation



This figure is generated from the retained protocol. Use the notebook to inspect all seeds and recompute the plotted quantities. The earlier lecture sections specify what the diagnostic can and cannot establish.